МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Направление подготовки: 10.03.01 Информационная безопасность
Образовательная программа: «Технологии защиты информации»

Дисциплина:
«Технологии и методы программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4
«Исследование и реверс-инжиниринг алгоритмов парольной защиты исполняемых файлов методами статического анализа»

Выполнил студент:

Пахомов Владимир Юрьевич / 

ФИО *Подпись*

Проверил:

Ищенко Алексей Петрович / _____

ФИО *Подпись*

_____

Отметка о выполнении

_____

Дата

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 АРХИТЕКТУРНЫЙ И СТРУКТУРНЫЙ АНАЛИЗ ИСПОЛНЯЕМОГО ФАЙЛА **4**

1.1 Первичная идентификация файла и среды сборки 4

1.2 Интерфейс пользователя и логика блокировки подбора 4

1.3 Анализ ресурсов форм в Resource Hacker 5

2 СТАТИЧЕСКИЙ АНАЛИЗ И ДЕКОМПИЛЯЦИЯ АЛГОРИТМА ВАЛИДАЦИИ **7**

2.1 Локализация процедур обработки событий в IDA Pro 7

2.2 Декомпиляция и исследование алгоритма Button1Click 7

2.3 Локализация эталонной строки сравнения unk_66963C 8

3 МАТЕМАТИЧЕСКОЕ ВОССТАНОВЛЕНИЕ ПАРОЛЯ И ВЕРИФИКАЦИЯ . . . **10**

3.1 Вывод формулы обратного преобразования 10

3.2 Посимвольный расчёт пароля 10

3.3 Практическая верификация решения 10

ЗАКЛЮЧЕНИЕ **11**

ВВЕДЕНИЕ

Цель работы: приобретение практических навыков исследования скомпилированных исполняемых модулей архитектуры x86 с использованием инструментария статического анализа, изучение низкоуровневой структуры оконных интерфейсов VCL, декомпиляция проприетарного алгоритма проверки пароля и аналитическое восстановление секретного значения без применения атак полного перебора и бинарной модификации исполняемого кода.

Постановка задачи:

1. Провести первичный анализ исполняемого файла формата PE32 с помощью утилит статической разведки (Detect It Easy) для определения компилятора и характеристик среды сборки.
2. Исследовать графический интерфейс приложения, зафиксировать значение открытого параметра («соли») и проанализировать логику блокировки при превышении лимита попыток ввода пароля.
3. Извлечь и структурировать ресурсы графических форм с помощью утилиты Resource Hacker с целью выявления идентификаторов элементов управления и сигнатур обработчиков событий.
4. В среде дизассемблера IDA Pro локализовать процедуры `FormShow` и `Button1Click`, выполнить их декомпиляцию в псевдокод и восстановить алгоритм валидации пароля.
5. Локализовать в сегменте данных программы эталонное значение, используемое для финального сравнения с результатом обработки пользовательского ввода.
6. Построить математическую модель обратного криптографического преобразования, выполнить посимвольный расчёт искомого пароля и осуществить практическую проверку корректности авторизации в приложении.

1 АРХИТЕКТУРНЫЙ И СТРУКТУРНЫЙ АНАЛИЗ ИСПОЛНЯЕМОГО ФАЙЛА

1.1 Первичная идентификация файла и среды сборки

На начальном этапе исследования был выполнен анализ структуры целевого исполняемого файла формата PE32. С помощью специализированной утилиты анализа бинарных файлов Detect It Easy были определены базовые характеристики исполняемого модуля (Рисунок 1).

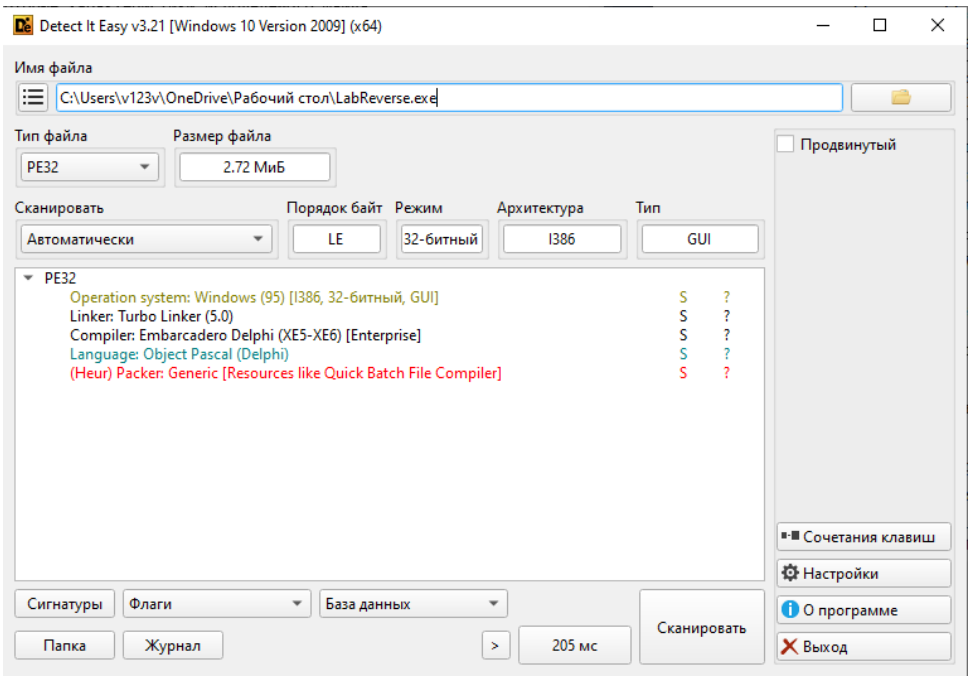


Рисунок 1 — Результаты анализа исполняемого файла в Detect It Easy

По результатам анализа установлено, что исследуемый файл представляет собой 32-разрядное приложение Windows (PE32), собранное компилятором Embarcadero C++ Builder.

1.2 Интерфейс пользователя и логика блокировки подбора

При штатном запуске программа инициализирует графический интерфейс (Рисунок 2), содержащий следующие элементы:

- Текстовое поле с индивидуально сгенерированным параметром «Секрет» (соль): `zj9b758v`;
- Поле для ввода парольной фразы;
- Кнопки управления «OK» (инициация процедуры проверки) и «Выход».

В случае ввода некорректного значения программа информирует пользователя диалоговым окном «Неверный пароль!». При превышении лимита попыток ввода (более двух неудачных попыток подряд) приложение переходит в состояние блокировки длительностью 10 секунд (Рисунок 3). Данный механизм исключает практическую возможность применения методов полного перебора (brute-force).

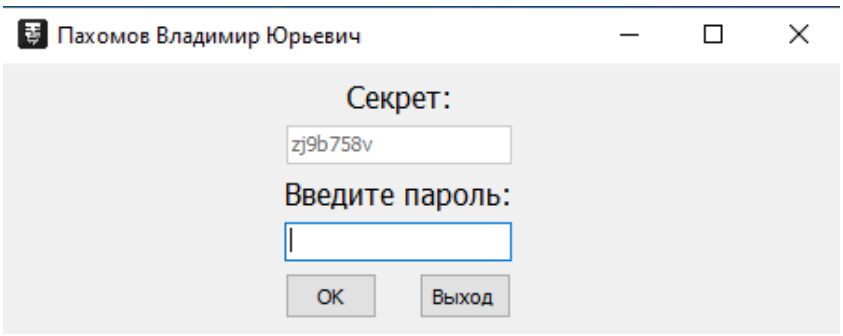


Рисунок 2 — Графический интерфейс исследуемого приложения

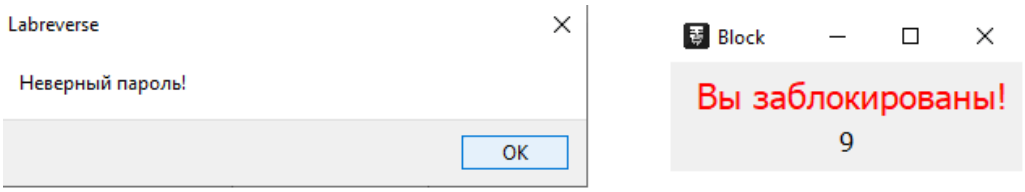


Рисунок 3 — Окна предупреждения об ошибке и временной блокировки ввода

1.3 Анализ ресурсов форм в Resource Hacker

С целью определения внутренней структуры оконных объектов и точек привязки событий был произведен разбор секции ресурсов `RCData` с помощью утилиты Resource Hacker. В результате анализа была извлечена декларация используемых форм:

```
object Form1: TForm1
  OnShow = FormShow
  object Label1: TLabel
    Caption = Введите' пароль:'
  end
  object Label2: TLabel
    Caption = Секрет':'
  end
  object Edit1: TEdit            // поле ввода пароля
  end
  object Button1: TButton
    Caption = 'OK'
    OnClick = Button1Click       // обработчик проверки пароля
  end
  object Edit2: TEdit            // поле отображения соли
    Enabled = False
  end
end

object Form2: TForm2             // форма успешной аутентификации
  Caption = 'Form2'
  object Label1: TLabel
    Caption = Поздравляем', вы подобрали пароль!'
  end
end
```

```
object Form3: TForm3              // форма блокировки при переборе
  Caption = 'Block'
  OnCloseQuery = FormCloseQuery
  OnShow = FormShow
  object Label1: TLabel
    Caption = Вы' заблокированы!'
  end
  object Timer1: TTimer
  object Timer2: TTimer
end
```

Полученное описание позволяет точно сопоставить элементы пользовательского интерфейса с обработчиками событий в бинарном коде: процедура `FormShow` отвечает за подготовку формы и отображение соли, а `Button1Click` реализует проверку введенных данных.

2 СТАТИЧЕСКИЙ АНАЛИЗ И ДЕКОМПИЛЯЦИЯ АЛГОРИТМА ВАЛИДАЦИИ

2.1 Локализация процедур обработки событий в IDA Pro

На основе имен методов в дизассемблере IDA Pro был выполнен поиск строковых ссылок в таблице методов формы. По адресу `0x0065DC8D` в секции данных `.data` располагаются дескрипторы соответствующих процедур:

```
.data:0065DC88                 db    0
.data:0065DC89                 dd offset sub_40330C
.data:0065DC8D aButton1click   db 12,'Button1Click'
.data:0065DC9A                 db  0Fh
.data:0065DC9B                 db    0
.data:0065DC9C                 dd offset sub_404400
.data:0065DCA0 aFormshow       db 8,'FormShow',0
```

Таким образом, установлены прямые адреса подпрограмм:

- `sub_40330C` — реализация обработчика нажатия кнопки проверки (`Button1Click`);
- `sub_404400` — подпрограмма инициализации окна и соли (`FormShow`).

2.2 Декомпиляция и исследование алгоритма Button1Click

Декомпиляция функции `sub_40330C` во внутреннее C-подобное представление выявила следующую логику работы алгоритма проверки:

```
void __fastcall sub_40330C(int a1)
{
  // Проверка соответствия длины пароля: ровно 8 символов
  v40 = sub_4042D4(v103) != 8;
  if ( v40 ) {
    // Вывод сообщения о некорректной длине строки
  } else {
    for ( j = 1; j < 9; ++j )  // Цикл по 8 символам пароля (1..8)
    {
      // Извлечение символа пароля и вычитание 48 код( '0')
      v53 = *(unsigned __int16 *)sub_404308((System::UnicodeString *)v94, j);
      v54 = v53 - 48;

      // Извлечение соответствующего символа соли и вычитание 48
      v48 = *(unsigned __int16 *)sub_404308((System::UnicodeString *)v86, j);
      v49 = v48 - 48;

      // Побитовое исключающее ИЛИ (XOR) нормализованных значений
      v42 = sub_4244FC(*v36) + v34;  // Результирующий байт XOR

      // Конвертация кода в символ через sub_4043BC и добавление в строку
      sub_4043BC(v66, v42);
      System::UnicodeString::operator+(v37, &unk_65D220, v66);
      sub_65B524(&unk_65D220, v65);
```

```
    }

    // Сравнение результирующей строки с эталоном unk_66963C
    if ( (unsigned __int8)sub_65B5C0(&unk_65D220, &unk_66963C) )
    {
      // Успешная валидация: отображение Form2
    }
    else
    {
      // Ошибка: инкремент счетчика неудачных попыток
      if ( ++dword_65D228 > 2 )
      {
        dword_65D228 = 0;
        // Запись состояния блокировки в системный реестр Windows
        System::Win::Registry::TRegistry::WriteString(v41, *v39);
        Vcl::Forms::TCustomForm::Show(*(Vcl::Forms::TCustomForm **)off_668E58);
    // Form3
      }
    }
  }
}
```

Анализ подпрограммы `sub_4043BC` показал, что она выполняет форматирование через `sprintf("%c", v42)`, то есть преобразует вычисленный числовой код в одиночный символ `Chr(v42)`.

Следовательно, для каждой позиции $i \in [0, 7]$ алгоритм выполняет элементарное математическое действие:

$$p_i = ord(password[i]) - 48, \quad s_i = ord(salt[i]) - 48, \quad t_i = p_i \oplus s_i$$

Полученная 8-символьная последовательность конкатенируется в накопителе `unk_65D220` и сравнивается с глобальной эталонной константой `unk_66963C`.

2.3 Локализация эталонной строки сравнения unk_66963C

С использованием перекрестных ссылок (XREF) к адресу `unk_66963C` была обнаружена функция инициализации `sub_40304C`:

```
.data:0066963C unk_66963C      db    ? ; DATA XREF: sub_40304C+14↑Ao
                                       ; sub_40330C:↑loc_40403Eo
```

Декомпилированный фрагмент `sub_40304C` демонстрирует последовательное копирование строк в глобальные переменные. В ячейку `unk_66963C` заносится строковый литерал по адресу `aN5p0d0a`:

```
sub_65B454(v9, aN5p0d0a);      // Загрузка эталонного литерала
sub_65B524(&unk_66963C, v9);   // Сохранение в целевую константу сравнения
```

При переходе по адресу метки `aN5p0d0a` в сегменте `.data` располагаются исходные байты строки:

```
.data:0065D29C                  db    0
.data:0065D29D aN5p0d0a         db 'N:5p0D0A',0          ; DATA XREF: sub_40304C
    ↑+132o
.data:0065D2A6 unk_65D2A6       db    6                  ; Следующая структура
    данных
```

Символ двоеточия `':'` является недопустимым в идентификаторах языка C/C++, вследствие чего автоматический анализатор IDA Pro опустил его при формировании имени метки (`aN5p0d0a`). Непосредственный анализ данных подтверждает, что эталон состоит ровно из 8 символов: **`N:5p0D0A`**.

3 МАТЕМАТИЧЕСКОЕ ВОССТАНОВЛЕНИЕ ПАРОЛЯ И ВЕРИФИКАЦИЯ

3.1 Вывод формулы обратного преобразования

На основе установленного прямого алгоритма получена аналитическая формула восстановления каждого символа пароля:

$$s_i = ord(salt[i]) - 48$$

$$p_i = ord(target[i]) \oplus s_i$$

$$password[i] = chr(p_i + 48)$$

3.2 Посимвольный расчёт пароля

В качестве исходных значений используются соль `salt = "zj9b758v"` и эталонная цепочка `target = "N:5p0D0A"`. Результаты пошагового вычисления сведены в таблицу 1.

Таблица 1 — Посимвольное восстановление пароля обратным XOR-преобразованием

i	salt[i]	$s_i = \text{ord} - 48$	target[i]	ord(target[i])	$p_i = \text{target} \oplus s$	Символ пароля ($p_i + 48$)
0	`'z'`	74	`'N'`	78	$78 \oplus 74 = 4$	$4 + 48 = 52 \rightarrow$ **4**
1	`'j'`	58	`':'`	58	$58 \oplus 58 = 0$	$0 + 48 = 48 \rightarrow$ **0**
2	`'9'`	9	`'5'`	53	$53 \oplus 9 = 60$	$60 + 48 = 108 \rightarrow$ **l**
3	`'b'`	50	`'p'`	112	$112 \oplus 50 = 66$	$66 + 48 = 114 \rightarrow$ **r**
4	`'7'`	7	`'0'`	48	$48 \oplus 7 = 55$	$55 + 48 = 103 \rightarrow$ **g**
5	`'5'`	5	`'D'`	68	$68 \oplus 5 = 65$	$65 + 48 = 113 \rightarrow$ **q**
6	`'8'`	8	`'0'`	48	$48 \oplus 8 = 56$	$56 + 48 = 104 \rightarrow$ **h**
7	`'v'`	70	`'A'`	65	$65 \oplus 70 = 7$	$7 + 48 = 55 \rightarrow$ **7**

Каждый вычисленный символ строго принадлежит разрешенному множеству алфавита (строчные латинские буквы и десятичные цифры). Итоговое восстановленное значение пароля:

40lrgqh7

3.3 Практическая верификация решения

Ввод вычисленного значения `40lrgqh7` в диалоговое окно программы привел к успешному прохождению валидации и открытию формы `Form2` с подтверждающим сообщением (Рисунок 4).

Программа подтвердила корректность пароля с первой попытки, что подтверждает правильность декомпилированного алгоритма и аналитических вычислений.

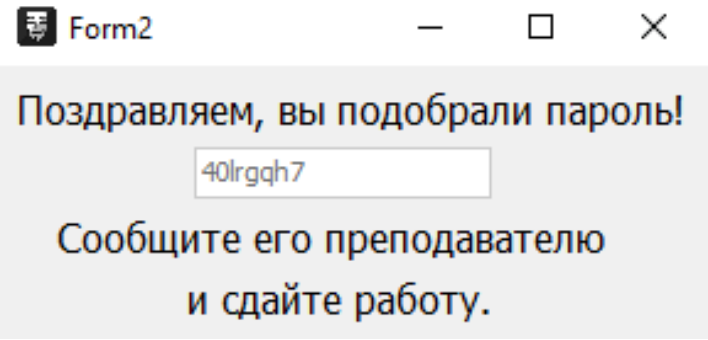


Рисунок 4 — Успешная авторизация в программе после ввода вычисленного пароля

ЗАКЛЮЧЕНИЕ

В процессе выполнения лабораторной работы были изучены и применены на практике методы статического реверс-инжиниринга исполняемых модулей PE32 без внесения изменений в исполняемый бинарный код.

Основные результаты работы:

1. С помощью утилит Detect It Easy и Resource Hacker определена среда разработки исследуемого модуля, извлечена структура оконных форм и выявлены точки входа ключевых обработчиков событий (`FormShow`, `Button1Click`).
2. В интерактивном дизассемблере IDA Pro исследована логика процедуры `sub_40330C`, реализующей проверку фиксированной 8-символьной длины пароля и его посимвольное побитовое XOR-преобразование с параметром «соли».
3. Локализована процедура `sub_40304C` и извлечено истинное значение 8-байтовой эталонной строки **`N:5p0D0A`** из секции `.data`, а также объяснена причина ее сокращенного именования (`aN5p0d0a`) в базе дизассемблера.
4. Выведена аналитическая формула обратного преобразования данных, на базе которой выполнен посимвольный расчёт искомого пароля.
5. Получен валидный пароль **`40lrgqh7`**, полностью удовлетворяющий критериям формата (строчные латинские буквы и цифры).
6. Выполнена успешная верификация пароля на учебном стенде без срабатывания механизмов блокировки попыток ввода.